

CareerIQ Database Design Guide

Explainable AI Career Intelligence and Skill Gap Analysis Platform

Document purpose	A simple guide to the MySQL Workbench database for the MCA project prototype.
Current scope	Candidate profiling, dummy job data, career matching, skill gaps, explanations, and learning roadmaps.
Important limitation	The prototype uses a curated dummy job dataset. It does not search live job portals or claim real-time market data.
Future scope	Live job APIs, market snapshots, and what-if simulation can be added after the core prototype is working.

Recommended build order: Profile → skills → career roles → dummy jobs → matching → gaps → roadmap.

1. How the database supports the project

The database keeps the candidate information separate from the master skill, role, and dummy job data. The analysis tables store the results each time a profile is evaluated. This makes the recommendations explainable and prevents old results from being overwritten.

The system compares **candidate_skills** with **career_role_skills** and **job_skills**. The result is saved as career recommendations, dummy job matches, readiness scores, skill gaps, and a learning roadmap.

*Relationship notation used in this document: **PK** = Primary Key (unique identity of a row). **FK** = Foreign Key (column that connects to another table).*

Database modules

Module	Tables	What it does
Candidate profile	users, candidate_profiles, education, experience, projects, certifications, resumes	Stores account and profile information.
Skills	skills, skill_aliases, candidate_skills	Stores standard skills and candidate proficiency.
Reference dataset	career_roles, career_role_skills, jobs, job_skills	Stores the manually curated careers and dummy jobs.
Analysis results	analysis_runs, career_recommendations, job_matches, match_explanations, readiness_scores, skill_gaps	Stores each result from the matching engine.
Learning	learning_resources, learning_resource_skills, learning_roadmaps, roadmap_steps	Stores recommended resources and ordered learning steps.

2. Final table list for the current prototype

These 25 tables are enough for the current dummy-data version. Do not add live-job integration tables at this stage.

No.	Table	Simple purpose
1	users	Account and login data
2	candidate_profiles	Candidate profile details
3	candidate_preferences	Target role and job preferences
4	education	Academic records
5	experience	Internship or work records
6	projects	Candidate projects
7	certifications	Courses and certificates
8	resumes	Uploaded resume and extracted text
9	skills	Master list of skills
10	skill_aliases	Alternative skill names for matching
11	candidate_skills	Candidate skill and proficiency
12	career_roles	Available career paths
13	career_role_skills	Skills required for each role
14	jobs	Curated dummy job opportunities
15	job_skills	Skills required for each dummy job
16	analysis_runs	One saved profile analysis
17	career_recommendations	Career role scores
18	job_matches	Dummy job scores
19	match_explanations	Why a score was given
20	readiness_scores	Career preparation score
21	skill_gaps	Missing or weak skills
22	learning_resources	Courses, videos, articles
23	learning_resource_skills	Skills taught by resources
24	learning_roadmaps	Personalized roadmap header
25	roadmap_steps	Ordered roadmap activities

3. Profile tables

One user creates one candidate profile. The profile can have many education, experience, project, certification, and resume records.

Table	PK	FK / relationship	What the table stores
users	user_id	—	Full name, email, password_hash, account role, active status. Email must be unique.
candidate_profiles	profile_id	user_id → users.user_id	Phone, city, LinkedIn URL, portfolio URL, profile summary. user_id must be unique here.
candidate_preferences	preference_id	profile_id → candidate_profiles.profile_id; target_role_id → career_roles.career_role_id	Preferred target role, location, work mode, expected salary.
education	education_id	profile_id → candidate_profiles.profile_id	Degree, college, specialization, CGPA/percentage, start/end year.
experience	experience_id	profile_id → candidate_profiles.profile_id	Company, designation, dates, experience description.
projects	project_id	profile_id → candidate_profiles.profile_id	Project title, description, GitHub/live URL, technologies.
certifications	certification_id	profile_id → candidate_profiles.profile_id	Certificate name, issuer, issue date, credential URL.
resumes	resume_id	profile_id → candidate_profiles.profile_id	File name, storage path, extracted resume text, parser status, upload date.

4. Skills and reference-data tables

Skills are stored only once in the skills master table. Other tables connect to it using skill_id. This avoids duplicate spellings and makes matching reliable.

Table	PK	FK / relationship	What the table stores
skills	skill_id	—	Master names such as Python, SQL, Excel, Power BI, Statistics, Machine Learning. normalized_name must be unique.
skill_aliases	alias_id	skill_id → skills.skill_id	Alternative names: PowerBI, MS Power BI, Python3. alias_name should be unique.
candidate_skills	candidate_skill_id	profile_id → candidate_profiles.profile_id; skill_id → skills.skill_id	Candidate proficiency 1–5, experience months, source (manual/resume), extraction confidence. Unique: profile_id + skill_id.
career_roles	career_role_id	—	Data Analyst, BI Analyst, Business Analyst, Junior Data Scientist. role_name should be unique.
career_role_skills	role_skill_id	career_role_id → career_roles.career_role_id; skill_id → skills.skill_id	Required level, importance weight, mandatory flag. Unique: career_role_id + skill_id.
jobs	job_id	career_role_id → career_roles.career_role_id	Curated dummy job title, sample company, location, work mode, description, experience requirement, is_demo_data, source.
job_skills	job_skill_id	job_id → jobs.job_id; skill_id → skills.skill_id	Skills required by each dummy job, required level, importance weight, mandatory flag. Unique: job_id + skill_id.

5. Matching and explainability tables

Each click on Analyze My Career creates one analysis_runs record. All results are connected to that analysis_id, so the system can show the history of results.

Table	PK	FK / relationship	What the table stores
analysis_runs	analysis_id	profile_id → candidate_profiles.profile_id; resume_id → resumes.resume_id	Analysis date, algorithm version, status, started and completed time.
career_recommendations	career_recommendation_id	analysis_id → analysis_runs.analysis_id; career_role_id → career_roles.career_role_id	Career match percentage, recommendation rank, summary explanation. Unique: analysis_id + career_role_id.
job_matches	job_match_id	analysis_id → analysis_runs.analysis_id; job_id → jobs.job_id	Dummy job match percentage, rank, summary explanation. Unique: analysis_id + job_id.
match_explanations	explanation_id	analysis_id → analysis_runs.analysis_id; job_match_id → job_matches.job_match_id OR career_recommendation_id → career_recommendations.career_recommendation_id	Detailed factors behind a score: matched skill, missing skill, contribution, explanation text. Fill only one of job_match_id or career_recommendation_id.
readiness_scores	readiness_score_id	analysis_id → analysis_runs.analysis_id; career_role_id → career_roles.career_role_id	Overall readiness score and components: skills, education, experience, projects.
skill_gaps	skill_gap_id	analysis_id → analysis_runs.analysis_id; career_role_id → career_roles.career_role_id; skill_id → skills.skill_id	Missing skill, current level, required level, priority score, status.

6. Learning roadmap tables

A roadmap converts the missing skills into practical learning steps. A learning resource can teach many skills, and one skill can be taught by many resources.

Table	PK	FK / relationship	What the table stores
learning_resources	resource_id	—	Course/video/article title, provider, resource type, URL, duration, free/paid flag.
learning_resource_skills	resource_id + skill_id	resource_id → learning_resources.resource_id; skill_id → skills.skill_id	Bridge table showing which resource teaches which skill. Composite primary key prevents duplicates.
learning_roadmaps	roadmap_id	analysis_id → analysis_runs.analysis_id; career_role_id → career_roles.career_role_id	Roadmap title, target career role, status.
roadmap_steps	roadmap_step_id	roadmap_id → learning_roadmaps.roadmap_id; skill_id → skills.skill_id; resource_id → learning_resources.resource_id	Step number, target skill, selected resource, expected days, completion status.

7. Main joins and relationship flow

These are the most important joins to explain during the viva. The arrow means the child table stores the foreign key.

Feature	Join condition	Why it is used
User profile	users.user_id = candidate_profiles.user_id	Shows the account together with the candidate profile.
Candidate skills	candidate_profiles.profile_id = candidate_skills.profile_id; candidate_skills.skill_id = skills.skill_id	Shows a candidate's skills with readable skill names.
Career requirements	career_roles.career_role_id = career_role_skills.career_role_id; career_role_skills.skill_id = skills.skill_id	Shows the required skills for a career role.
Dummy job requirements	jobs.job_id = job_skills.job_id; job_skills.skill_id = skills.skill_id	Shows the required skills for a sample job.
Career result	analysis_runs.analysis_id = career_recommendations.analysis_id; career_recommendations.career_role_id = career_roles.career_role_id	Shows career recommendations produced by an analysis.
Job result	analysis_runs.analysis_id = job_matches.analysis_id; job_matches.job_id = jobs.job_id	Shows dummy job opportunities ranked for the candidate.
Skill gap	skill_gaps.skill_id = skills.skill_id; skill_gaps.career_role_id = career_roles.career_role_id	Shows what skill is missing for which target role.
Roadmap	learning_roadmaps.roadmap_id = roadmap_steps.roadmap_id; roadmap_steps.skill_id = skills.skill_id	Shows the ordered skills a candidate should learn.

Basic matching logic: compare candidate_skills with career_role_skills and job_skills. Matched skills increase the score. Missing mandatory skills create skill_gaps and reduce the score.

8. Dummy dataset for the prototype

Use a small but realistic dataset. It is enough to show matching, explanations, gaps, and roadmaps without claiming live job search.

Career roles and required sample skills

Career role	Suggested skills
Data Analyst	SQL, Excel, Python, Pandas, Statistics, Data Visualization
Business Analyst	Excel, SQL, Power BI, Communication, Problem Solving
BI Analyst	SQL, Power BI, Tableau, Excel, Data Visualization
Junior Data Scientist	Python, SQL, Pandas, Statistics, Machine Learning, Scikit-learn

Sample jobs

Job title	Sample company	Career role	Demo data rule
Junior Data Analyst	Insight Analytics Pvt Ltd	Data Analyst	is_demo_data = TRUE
Data Analyst Intern	DataSphere Solutions	Data Analyst	is_demo_data = TRUE
Business Analyst Trainee	Nexa Consulting	Business Analyst	is_demo_data = TRUE
Junior BI Analyst	Vision Metrics	BI Analyst	is_demo_data = TRUE
Reporting Analyst	Alpha Systems	BI Analyst	is_demo_data = TRUE
Data Science Intern	AI Labs India	Junior Data Scientist	is_demo_data = TRUE
Junior Data Scientist	PredictiveWorks	Junior Data Scientist	is_demo_data = TRUE
MIS Executive	SmartOps Pvt Ltd	Business Analyst	is_demo_data = TRUE

Important viva statement: “The current prototype uses a curated MCA Project Dataset. Live job-portal integration is future work.”

9. Future enhancement tables

Do not build these tables now. Add them only after the core prototype is complete.

Future table	Purpose	When to add it
market_skill_demand	Stores periodic demand snapshots for skills, job counts, role, period, and source.	When real or regularly updated market data is available.
what_if_simulations	Stores a scenario such as “What if I learn Machine Learning?” and the new expected match score.	After normal matching and roadmaps are complete.
simulation_skills	Stores the skills added or upgraded in a what-if scenario.	Only together with what_if_simulations.
job_sources	Stores job portal/API source information and synchronization settings.	Only when official external APIs are approved.
job_import_logs	Stores details of a data import, source date, success/failure, and number of jobs added.	Only when automatic external imports are added.

10. Implementation rules for MySQL Workbench

Rule	Simple explanation
Use InnoDB	It supports foreign keys and transactions.
Use UTF-8	Use utf8mb4 for names, descriptions, and resume text.
Use foreign keys	Create every FK shown in this guide to maintain correct relationships.
Use unique constraints	Use them on users.email, skills.normalized_name, and bridge table pairs.
Use DECIMAL for scores	Use DECIMAL(5,2) for values such as 89.00 percent.
Use TINYINT for levels	Store proficiency and required levels from 1 to 5.
Do not store plain passwords	Use password_hash only.
Store resume path, not BLOB	Keep the file in storage and save its path/URL in resumes.
Avoid deleting master data	Do not delete a skill or career role if related records exist; mark it inactive if needed.

Conclusion

The 25-table core database is sufficient for the current MCA prototype. It demonstrates an explainable career recommendation system using controlled dummy data, while keeping live job search and real-time market intelligence as future enhancements.